
LSST Astronomical Time-Series Classification: Benchmarking Advanced Sequential Deep Learning

A Final Technical Report Submitted in Partial Fulfillment of the
Requirements for DS 5500: Capstone — Applications in Data Science

Authors: Chirag Mahaveer Chivate
Ashwin Praveen Khairnar
Rohan Kumar Ojha

Team: Team 18

Instructor: Dr. Fatema Nafa

Course: DS 5500 — Capstone: Applications in Data Science

Institution: Northeastern University, Khoury College of Computer Sciences

Date: April 12, 2026

GitHub Repo: [https://github.com/DS-Capstone-Northeastern/
LSST_Classification](https://github.com/DS-Capstone-Northeastern/LSST_Classification)

Abstract

Background: The upcoming Vera C. Rubin Observatory’s Legacy Survey of Space and Time (LSST) will generate millions of transient alerts nightly, requiring robust automated classifiers that can handle sparse, irregularly sampled multi-passband light curves with extreme class imbalance.

Objective: This project benchmarks advanced sequential deep learning architectures—specifically Temporal Fusion Transformers (TFT), pure State Space Models (SSM), and a hybrid SSM + MLP architecture—against original Kaggle competition baselines on the unblinded PLAsTiCC dataset comprising approximately 3.5 million astronomical objects across 14 primary target classes.

Methods: All three architectures share a standardized stratified data split (100 k train / 15 k validation / 3.39 M test) and a common evaluation script computing PLAsTiCC Weighted Log-Loss, Macro F1, Macro PR-AUC, and Multiclass Brier Score. The TFT uses Gated Residual Networks and multi-head attention with Optuna Bayesian hyperparameter optimization; the pure SSM employs per-passband Kalman/RTS smoothing concatenated with XGBoost; and the hybrid fuses Kalman-derived light-curve features with MLP-encoded static metadata into a HistGradientBoosting classifier.

Results: The hybrid SSM + MLP model achieves a PLAsTiCC Weighted Log-Loss of **0.1050**, dramatically outperforming the Kaggle first-place baseline of 0.6850. It also records the highest Macro F1 (0.6947) and Macro PR-AUC (0.7489). The pure SSM achieves 0.5177 Weighted Log-Loss with 83.95% accuracy, while the TFT attains 0.4080 Weighted Log-Loss.

Conclusion: Fusing structured state-space representations of intermittent light curves with MLP-encoded static metadata yields massive predictive gains over classical feature-engineered approaches and standalone deep learning models for astronomical time-series classification.

Keywords: astronomical classification, PLAsTiCC, state space models, temporal fusion transformer, irregular time-series

Contents

1	Introduction	5
1.1	Problem Statement	5
1.2	Motivation and Significance	5
1.3	Research Questions / Objectives	6
1.4	Report Structure	6
2	Related Work	6
2.1	The PLAsTiCC Challenge and Classical Approaches	6
2.2	Attention-Based Models for Time-Series	6
2.3	State Space Models for Long Sequences	7
2.4	Gradient Boosting and Ensemble Methods	7
2.5	Positioning of This Work	7
3	Data Pipeline and Preprocessing	7
3.1	Dataset Description	8
3.2	Exploratory Data Analysis (EDA)	8
3.3	Data Cleaning and Preprocessing	9
3.3.1	Leakage Prevention	9
3.3.2	Missing Value Strategy	9
3.3.3	Outlier Handling	9
3.4	Feature Engineering and Transformation	9
3.5	Train / Validation / Test Split	10
3.6	Pipeline Pseudo-code	10
4	Model Selection and Implementation	11
4.1	Model Family Justification	11
4.2	Model Architecture	12
4.2.1	Branch A: Temporal Fusion Transformer (TFT)	12
4.2.2	Branch B: Pure State Space Model (SSM)	12
4.2.3	Branch C: Hybrid SSM + MLP	13
4.3	Coding Paradigm and Design Architecture	14
4.3.1	Paradigm Choice	14
4.3.2	Module Structure	14
4.4	Hyperparameters	15
4.5	Training Procedure Pseudo-code	15
4.6	Fine-tuning / Prompt Engineering	16
5	Results and Model Evaluation	16

5.1	Evaluation Metrics	16
5.2	Baseline Comparison	17
5.3	Learning Curves and Training Dynamics	17
5.4	Error Analysis	17
5.5	Ablation Study	18
6	Testing and Validation	18
6.1	Testing Framework and Strategy	18
6.2	Unit Test Cases — Data Pipeline	19
6.3	Unit Test Cases — Model	20
6.4	Integration Test Cases — End-to-End Pipeline	21
7	GitHub Usage and Version Control	21
7.1	Repository Information	21
7.2	Branching Strategy	21
7.3	Team Contribution Summary	22
7.4	Commit History Excerpt	23
7.5	Pull Request and Code Review Evidence	23
8	Deployment and Reproducibility	24
8.1	Environment and Dependencies	24
8.2	Reproduction Steps	24
8.3	Compute and Kaggle Workflow	25
8.4	Demo Application	25
9	Discussion	25
9.1	Interpretation of Results	25
9.2	Limitations	26
9.3	Ethical Considerations	26
10	Conclusion and Future Work	26
10.1	Summary of Contributions	27
10.2	Future Work	27
Appendix A	Full Pseudo-code Listing	29
Appendix B	Extended Test Case Log	30
Appendix C	Individual Contribution Statement	31

List of Figures

1	Exploratory data analysis: class distribution, temporal gap distribution, and per-object sequence length statistics across the PLAsTiCC unblinded universe.	9
2	End-to-end pipeline architecture. Shared unblinded PLAsTiCC data is routed through three distinct modeling paradigms (TFT, Pure SSM, Hybrid) before converging for standardized evaluation.	13
3	Training dynamics and model comparison.	17
4	GitHub branch and merge history illustrating the three-tier workflow: Feature → Dev → Main.	22
5	Recent project commit history as displayed in the GitHub repository.	23

List of Tables

1	Dataset summary	8
2	Data partition sizes	10
3	Model family comparison for astronomical time-series classification	11
4	Hyperparameter configurations for all three architectures	15
5	Evaluation metrics and justification	16
6	PLAsTiCC Weighted Log-Loss comparison (lower is better)	17
7	Additional performance metrics on the test set	17
8	Unit test cases for the data preprocessing pipeline	19
9	Unit test cases for the model modules	20
10	Integration test cases for the full pipeline	21
11	Individual GitHub contribution summary	22
12	Selected pull requests demonstrating team collaboration	23

1. Introduction

1.1 Problem Statement

The upcoming Vera C. Rubin Observatory’s Legacy Survey of Space and Time (LSST) will map the observable sky every few days, generating an unprecedented influx of time-domain astronomical data. Each night, the survey is expected to produce millions of transient alerts corresponding to supernovae, variable stars, active galactic nuclei, and other time-varying phenomena. Processing this heterogeneous data stream requires robust, automated algorithms capable of classifying transients and variable stars in near real-time.

The primary challenge lies in the nature of the observational data itself. Astronomical light curves—time-series recordings of an object’s brightness across multiple optical passbands—are inherently *sparse* and *irregularly sampled*. Significant temporal gaps (Δt) arise due to weather, seasonal invisibility, and telescope scheduling constraints. Furthermore, the training data available for supervised learning is profoundly *non-representative*: common phenomena such as Type Ia Supernovae dominate the universe, while scientifically critical rare events like Kilonovae have extremely limited representation.

This project tackles this highly imbalanced, multimodal, multiclass time-series classification problem using the unblinded PLAsTiCC dataset (PLAsTiCC Team, 2018). The objective is to accurately map a combination of 26 static metadata features (e.g., host galaxy photometric redshift, Milky Way extinction) and dynamic, irregularly sampled 6-band (u, g, r, i, z, y) light-curve sequences to distinct astronomical target classes.

1.2 Motivation and Significance

Solving this classification problem is of paramount importance to the astronomical community. The LSST will generate a data onslaught that far exceeds human processing capacity. Automated classifiers act as real-time filters for the observatory: distinguishing rare, fast-fading events (e.g., Kilonovae, novel anomalies) from common phenomena the moment they are detected allows global robotic telescopes to prioritize their limited follow-up resources effectively.

Traditional recurrent neural networks (RNNs/LSTMs) often fail to maintain memory across some massive, irregular time gaps characteristic of astronomical data. This motivates the exploration of architectures specifically designed to handle such intermittency—namely Temporal Fusion Transformers, which leverage attention mechanisms, and State Space Models, which provide continuous-time dynamical representations. Success would provide the astronomical community with a deployable pipeline for next-generation survey operations.

1.3 Research Questions / Objectives

- RQ1.** Can attention-based Temporal Fusion Transformers effectively capture long-range dependencies in irregularly sampled, multi-passband astronomical light curves?
- RQ2.** Do classical State Space Models (Kalman filter / RTS smoother) provide competitive fixed-length feature representations compared to learned deep sequential architectures?
- RQ3.** Does a hybrid architecture—fusing SSM-derived light-curve features with MLP-encoded static metadata—outperform both standalone approaches and the original Kaggle competition baselines?

1.4 Report Structure

This report is organized as follows: Section 2 reviews related work; Section 3 describes the data pipeline; Section 4 details model selection and implementation; Section 5 presents results and evaluation; Section 6 covers testing and validation; Section 7 documents version control and team contributions; Section 8 addresses deployment and reproducibility; and Section 10 concludes with future work.

2. Related Work

2.1 The PLAsTiCC Challenge and Classical Approaches

The Photometric LSST Astronomical Time-Series Classification Challenge (PLAsTiCC) (PLAsTiCC Team, 2018) was a community-wide competition designed to spur the development of automated classifiers in preparation for the LSST data stream. Over 1 000 teams participated, and the results (Hložek et al., 2020) demonstrated that the most successful solutions relied heavily on extensive manual feature engineering combined with gradient-boosted tree ensembles (LightGBM, XGBoost, CatBoost). The winning solution by Boone (2019) used Gaussian Process light-curve fitting with data augmentation, achieving a private leaderboard Weighted Log-Loss of 0.6850. A critical finding was that success depended heavily on the inclusion of host galaxy photometric redshift as a feature, and that all classifiers struggled to identify novel “Other” class objects. The unblinded data release (Kessler et al., 2019) later provided ground-truth labels for the full 3.5M-object universe, enabling the rigorous evaluation conducted in this project.

2.2 Attention-Based Models for Time-Series

The Temporal Fusion Transformer (TFT) (Lim et al., 2021) was introduced for multi-horizon time-series forecasting, combining Variable Selection Networks, Gated Residual Networks, and interpretable multi-head attention to handle heterogeneous inputs—static metadata, past observed

values, and known future inputs. While designed for forecasting rather than classification, the architecture’s ability to fuse static context with temporal dynamics makes it a natural candidate for astronomical data where static metadata (redshift, extinction) must be integrated with variable-length light curves. Standard Transformers suffer from $O(N^2)$ memory scaling (Vaswani et al., 2017), but the TFT mitigates this through its variable selection mechanism and gated architecture.

2.3 State Space Models for Long Sequences

Structured State Spaces (S4) (Gu et al., 2022) address the fundamental limitation of Transformers on very long sequences by providing linear-time, linear-memory sequence modeling through continuous-time dynamical systems. The HiPPO initialization and NPLR parameterization enabled S4 to achieve state-of-the-art results on the Long Range Arena benchmark, solving tasks (e.g., Path-X at length 16 384) where all prior architectures failed. The Mamba architecture (Gu and Dao, 2023) subsequently introduced selective state spaces with hardware-aware design for even more efficient sequence modeling. For astronomical applications, classical Kalman filtering—a specific instance of linear state-space inference—offers an interpretable, physics-grounded approach to extracting features from irregularly sampled time-series without the computational overhead of neural sequence models. The hybrid approach of combining SSM-based feature extraction with downstream classifiers was motivated by recent work on metadata-aware time-series foundation models (Quan et al., 2024).

2.4 Gradient Boosting and Ensemble Methods

XGBoost (Chen and Guestrin, 2016) and HistGradientBoosting (scikit-learn) represent the state of the art for tabular classification and were the backbone of most top-performing PLAsTiCC solutions. These methods handle heterogeneous features, missing values, and class imbalance effectively, making them ideal downstream classifiers for hand-crafted or SSM-derived feature vectors.

2.5 Positioning of This Work

Our project differs from prior PLAsTiCC solutions in three key ways. First, we evaluate the full unblinded 3.5M-object universe rather than the competition’s partial test set. Second, we benchmark modern deep sequential architectures (TFT, SSM) that did not exist at the time of the original competition. Third, our hybrid SSM + MLP approach eliminates the need for Gaussian Process fitting or manual template-based feature engineering by delegating time-series feature extraction to a Kalman filter and static feature extraction to a neural network, fusing both representations in a gradient-boosted classifier.

3. Data Pipeline and Preprocessing

3.1 Dataset Description

Table 1: Dataset summary

Dataset	Source	Samples	Features	License
PLAsTiCC Metadata	PLAsTiCC Team (2018), Zenodo	~3.5M objects	26 static cols	Public
PLAsTiCC Light Curves	12 CSV files (22 GB total)	~453M rows	6 dynamic cols	Public

The PLAsTiCC universe comprises approximately 3.5 million distinct astronomical objects, generated using the SuperNova ANALysis (SNANA) simulation package to mimic observations from the upcoming LSST (PLAsTiCC Team, 2018). The unblinded release (Kessler et al., 2019) provides definitive ground-truth targets and redshifts for all objects. The data is multimodal:

- **Static Metadata** (26 columns): Time-invariant physical properties including sky coordinates (`ra`, `decl`), host galaxy photometric redshift (`hostgal_photoz`), distance modulus (`distmod`), and Milky Way Galactic extinction (`mwebv`).
- **Dynamic Light Curves** (6 columns): Irregularly sampled brightness measurements (`flux`, `flux_err`) across 6 optical passbands (`u`, `g`, `r`, `i`, `z`, `y`), indexed by Modified Julian Date (`mjd`).

The dataset is heavily imbalanced: common phenomena like Type Ia Supernovae (Target 90) dominate with millions of samples, while rare events such as Kilonovae (Target 64) have extremely limited representation. The unblinded release resolves the original competition’s “Other” class (Target 99) into finer sub-categories (991–994), yielding 17 resolved target classes.

3.2 Exploratory Data Analysis (EDA)

Exploratory analysis was conducted in a shared notebook (`00_data_exploration.EDA.ipynb`) across the full 3.5M-object universe. Key findings included severe temporal intermittency with observation gaps (Δt) spanning days to months, extreme class imbalance (the majority class exceeds the minority class by several orders of magnitude), and significant variation in sequence lengths across objects and passbands.

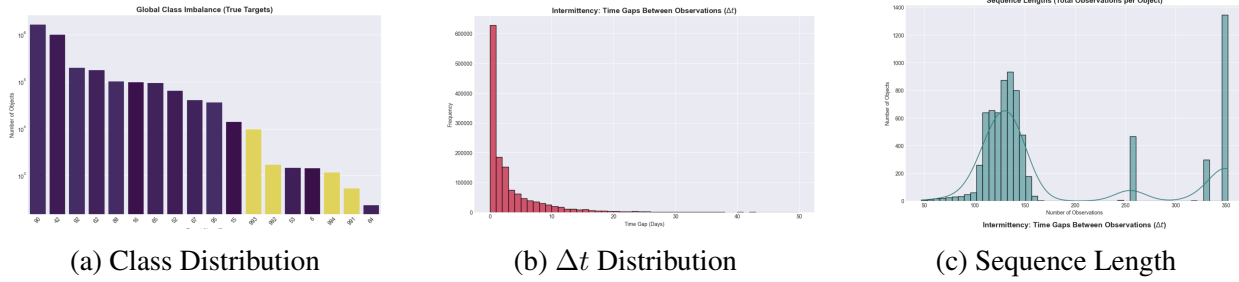


Figure 1: Exploratory data analysis: class distribution, temporal gap distribution, and per-object sequence length statistics across the PLAsTiCC unblinded universe.

3.3 Data Cleaning and Preprocessing

3.3.1 Leakage Prevention

Simulation truth columns susceptible to target leakage—all columns with prefixes `true_` and `sim_`—were explicitly excluded from all model inputs. Only leak-safe metadata columns were retained.

3.3.2 Missing Value Strategy

For static metadata, missing numerical values were imputed with the column median (computed from training data only). The `StaticMetadataPreprocessor` class encapsulates this logic, storing training-set medians and applying them consistently to validation and test splits to prevent data leakage.

3.3.3 Outlier Handling

Rather than removing outliers—which in astronomy may correspond to genuine astrophysical phenomena such as supernova peak brightness—we applied `RobustScaler` (based on median and interquartile range) to both dynamic flux values and static metadata features. This approach is immune to extreme magnitude spikes while preserving astrophysically meaningful signals.

3.4 Feature Engineering and Transformation

Feature engineering was customized for each architectural branch:

Shared Pipeline: A continuous time-gap feature (Δt) was calculated between consecutive observations per object, and a `time_since_start` feature was computed relative to the first observation. Both were generated in `preprocessing.py` via the `engineer_temporal_features()` function.

TFT Branch: Dynamic continuous features (`flux`, `flux_err`, `delta_t`, `time_since_start`) were scaled with `RobustScaler`. Static features (`hostgal_photoz`, `hostgal_photoz_err`, `mwebv`) were similarly scaled. Passband was treated as a categorical feature embedded via `nn.Embedding`. All sequences were padded to a maximum length of 350 steps with boolean attention masks so the Transformer ignores artificial padding.

SSM Branch: Per-passband Kalman filter / RTS smoother was applied to raw light curves, extracting 8 basic features per passband (48 total) and 25 global features, yielding a 73-dimensional fixed-length feature vector per object. Static metadata was imputed and standardized via `StaticMetadataPreprocessor`.

Hybrid Branch: A separate Kalman feature extraction (`kalman_features_for_object()`) produced 10 per-passband features $\times$ 6 passbands = 60, plus 40 aggregated statistics and 11 summary statistics, totaling 111 light-curve features per object. These were concatenated with 9 leak-safe static metadata features and 32-dimensional MLP embeddings to form a 152-dimensional hybrid vector.

3.5 Train / Validation / Test Split

To ensure all models are evaluated on the exact same data, a stratified split based on the true target class was computed once in a shared notebook (`01_stratified_data_split.ipynb`) and exported as lightweight ID ledgers serving as the “Source of Truth.”

Table 2: Data partition sizes

Split	Samples	Percentage	Notes
Training	100 000	2.86%	Stratified by target class
Validation	15 000	0.43%	Stratified; held-out
Test	3 385 738	96.71%	Never seen during training
Total	3 500 738	100%	

3.6 Pipeline Pseudo-code

Pseudo-code: End-to-End Data Preprocessing Pipeline (TFT Branch)

Require: Raw light-curve CSVs $\mathcal{L}$, metadata CSVs $\mathcal{M}$, ID ledger $\mathcal{I}$

Ensure: Padded, masked PyTorch tensors ($\mathcal{D}_{train}$, $\mathcal{D}_{val}$)

- 1: Load valid object IDs from $\mathcal{I}$ (e.g., `train_ids.csv`)
- 2: **Chunked loading:** iterate 22 GB CSVs in 500 k-row chunks; retain only valid IDs
- 3: Concatenate filtered chunks for light curves and metadata
- 4: **for** each object in light curves **do**
- 5: Sort observations chronologically by `mjd`

```

6:   Compute time_since_start = mjd - min(mjd)
7:   Compute  $\Delta t$  via consecutive mjd differences; fill first with 0
8: end for
9: Apply RobustScaler to flux, flux_err (fit on training split only)
10: Apply RobustScaler to static columns (fit on training split only)
11: Merge static metadata into dynamic table on object_id
12: for each object do
13:   Extract static vector  $\in \mathbb{R}^3$ , dynamic continuous  $\in \mathbb{R}^{T \times 4}$ , passband  $\in \mathbb{Z}^T$ 
14:   Truncate or pad to  $T_{\max} = 350$ ; generate boolean attention mask
15: end for
16: return PyTorch DataLoader with WeightedRandomSampler for class balance

```

4. Model Selection and Implementation

4.1 Model Family Justification

Table 3: Model family comparison for astronomical time-series classification

Family	Strengths	Limitations	Selected?
Classical ML (Feature Eng. + Gradient Boosting)	Interpretable, fast, handles heterogeneous features	Requires manual feature engineering; limited temporal modeling	Baseline
TFT (Attention-Based DL)	Fuses static/dynamic data; interpretable attention; end-to-end	$O(N^2)$ attention cost; large data requirement	✓
SSM (Kalman + Classifier)	Physics-grounded; handles irregular sampling naturally; linear-time	Requires domain knowledge for feature design	✓
Hybrid (SSM + MLP)	Combines strengths of both paradigms; interpretable features	Multi-stage pipeline complexity	✓

Given the multimodal nature of the data (static metadata + irregular time-series), extreme class imbalance, and the need to process 3.5M objects within constrained compute budgets, we selected three complementary architectures. The TFT tests whether attention-based end-to-end deep learning can handle the irregular sampling directly. The pure SSM tests whether classical signal processing (Kalman filtering) can extract competitive features without GPU-based training. The hybrid fuses both paradigms to test whether complementary representations yield additive gains.

4.2 Model Architecture

4.2.1 Branch A: Temporal Fusion Transformer (TFT)

The TFT architecture (Lim et al., 2021) was adapted from a forecasting model to a classification model by replacing the continuous output head with a multi-class classification head. The key components are:

- **Feature Embeddings:** Passband (0–5) is embedded via `nn.Embedding` into d_{model} -dimensional space. Continuous static features (3 dims) and dynamic features (4 dims) are each projected through Gated Residual Networks (GRNs) into d_{model} dimensions.
- **Dynamic Feature Combiner:** The continuous and categorical dynamic embeddings are concatenated and passed through another GRN to produce a unified sequence representation.
- **Static Context Injection:** The encoded static context vector is added (broadcast) to every time step of the sequence representation.
- **Transformer Encoder:** A standard `nn.TransformerEncoder` with n_{head} attention heads and n_{layers} encoder layers processes the enriched sequence. Padding positions are masked via `src_key_padding_mask`.
- **Classification Head:** Max-pooling across the time dimension (ignoring padded positions) produces a fixed-length sequence representation, which is concatenated with the static context and passed through a two-layer MLP with ReLU activation to produce class logits.

4.2.2 Branch B: Pure State Space Model (SSM)

The pure SSM branch applies a classical 2D constant-velocity Kalman filter / Rauch–Tung–Striebel (RTS) smoother independently to each of the 6 passbands per object. The state vector $\mathbf{x}_t = [f_t, v_t]^\top$ models latent flux f_t and its rate of change v_t . The transition model uses scaled time increments $\Delta t_s = \Delta t / \text{dt_scale}$, where `dt_scale` is set to the global median positive inter-epoch ΔMJD from training data (≈ 3.00). For each passband, 8 basic features are extracted (peak flux, time to peak, rise/decline rates, asymmetry, amplitude, duration, and an observation indicator). An additional 25 global features are computed from concatenated passband trajectories, including velocity statistics, acceleration, flux variance, skewness, kurtosis, signal-to-noise ratio, exponential decay rate, $g-r$ color proxies, and multi-band correlation. The resulting 73-dimensional SSM

feature vector is concatenated with imputed and standardized leak-safe static metadata and classified by an XGBoost model (histogram-based, 600 estimators, max depth 8, learning rate 0.05).

4.2.3 Branch C: Hybrid SSM + MLP

The hybrid architecture delegates processing to specialized sub-components:

- **MLP Branch:** A `scikit-learn MLPClassifier` with hidden layers (64, 32) and ReLU activation is trained on 9 leak-safe static metadata features via 5-fold out-of-fold (OOF) cross-validation. The last hidden layer (32 dimensions) is extracted as a learned static embedding.
- **Kalman Branch:** A 2D local linear trend Kalman filter is applied per-passband, extracting 10 features per band (innovation statistics, final level/trend, observation count) plus 40 aggregated statistics and 11 flux summary statistics, yielding 111 features per object.
- **Fusion:** The 9-dimensional scaled metadata, 32-dimensional MLP embedding, and 111-dimensional Kalman features are concatenated into a 152-dimensional hybrid vector.
- **Final Classifier:** A `HistGradientBoostingClassifier` (max depth 6, learning rate 0.05, 400 iterations) is trained on the hybrid vector with 5-fold stratified cross-validation, producing OOF predictions for training and averaged fold predictions for test.

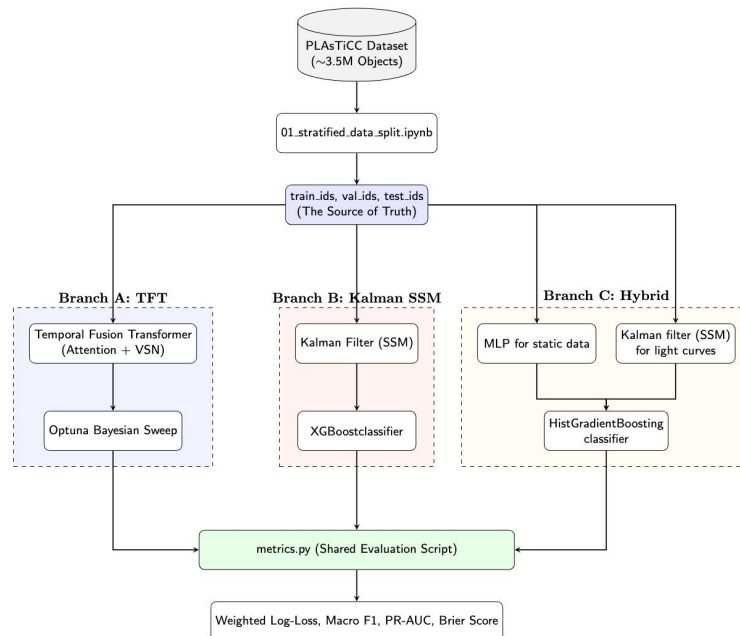


Figure 2: End-to-end pipeline architecture. Shared unblinded PLAsTiCC data is routed through three distinct modeling paradigms (TFT, Pure SSM, Hybrid) before converging for standardized evaluation.

4.3 Coding Paradigm and Design Architecture

4.3.1 Paradigm Choice

The project employs a hybrid of Object-Oriented Programming (OOP) and functional programming. PyTorch `nn.Module` subclasses define all neural network components (e.g., `GatedResidualNetwork`, `TFTClassifier`, `PLAsTiCCTFTDataset`). Classical pipeline components use a mix of data-classes (`KalmanRTSResult`), OOP (`StaticMetadataPreprocessor`), and pure functions (e.g., `kalman_filter_rts`, `extract_ssm_features_matrix`).

4.3.2 Module Structure

```

1 LSST_Classification/
2 |-- 00_Shared_Pipeline/
3 |   |-- 00_data_exploration_EDA.ipynb
4 |   |-- 01_stratified_data_split.ipynb
5 |-- data_splits/
6 |   |-- train_ids.csv           # 100k object IDs
7 |   |-- val_ids.csv             # 15k object IDs
8 |   |-- test_ids.csv            # ~3.39M object IDs
9 |-- TFT/
10 |   |-- preprocessing.py        # Chunked loading, RobustScaler, delta_t
11 |   |-- dataset.py              # PLAsTiCCTFTDataset (padding, masking)
12 |   |-- model_architecture.py   # GRN, TFTClassifier
13 |   |-- metrics.py              # Shared evaluation (WLL, Macro F1, etc.)
14 |   |-- 01-train-tft.ipynb      # Kaggle training notebook
15 |-- SSM/
16 |   |-- ssm/
17 |   |   |-- ssm_kalman.py        # Kalman filter + RTS smoother
18 |   |   |-- ssm_features.py     # 73-dim feature extraction
19 |   |   |-- static_metadata.py  # Leak-safe metadata preprocessor
20 |   |-- train_ssm_split.py      # XGBoost training script
21 |-- ssm_mlp/
22 |   |-- hybrid_plasticc_pipeline_full.ipynb
23 |-- literature_survey/          # 3 core research papers
24 |-- Metrics_visuals.ipynb       # Comparison visualizations
25 |-- project_tracker.xlsx        # Task management
26 |-- README.md

```

Listing 1: Repository directory structure

4.4 Hyperparameters

Table 4: Hyperparameter configurations for all three architectures

Parameter	TFT	Pure SSM	Hybrid
Learning rate	1.48×10^{-4}	0.05 (XGB)	0.05 (HGB)
Batch size	64	Full matrix	Full matrix
Epochs / Iterations	15 (Optuna trials)	600 trees	400 iterations
Optimizer	AdamW	—	—
Dropout rate	0.375	—	—
d_{model}	128	—	—
Attention heads	8	—	—
Encoder layers	3	—	—
Weight decay	2.60×10^{-4}	—	—
Max depth	—	None (RF) / 8 (XGB)	6
Subsample	—	0.8	—
Kalman q	—	0.01	10^{-3} (q_{level})
Kalman r_{min}	—	10^{-3}	10^{-3}
dt_scale	—	≈ 3.00 (auto)	—

4.5 Training Procedure Pseudo-code

Pseudo-code: TFT Training Loop with Optuna Bayesian Optimization

Require: $\mathcal{D}_{\text{train}}, \mathcal{D}_{\text{val}}$, Optuna study with $n_{\text{trials}} = 10$

Ensure: Best model checkpoint $\mathcal{M}^*$ minimizing PLAsTiCC Weighted Log-Loss

- 1: Create Optuna study with MEDIANPRUNER($n_{\text{startup}} = 2, n_{\text{warmup}} = 3$)
- 2: **for** each trial $t = 1$ to n_{trials} **do**
- 3: Sample: $d_{\text{model}} \in \{64, 128, 256\}, n_{\text{head}} \in \{4, 8\}, n_{\text{layers}} \in \{2, 3\}$
- 4: Sample: dropout $\in [0.2, 0.4], \text{lr} \in [10^{-4}, 10^{-3}]$ (log-uniform)
- 5: Initialize TFTClassifier; set ADAMW optimizer, COSINEANNEALINGWARMRESTARTS scheduler
- 6: Set custom CROSSENTROPYLOSS with $2\times$ weight on classes 64 and 99
- 7: **for** epoch = 1 to 15 **do**
- 8: Train with BFloat16 mixed precision; clip gradients (max norm = 1.0)
- 9: Compute validation PLAsTiCC Weighted Log-Loss
- 10: **if** val_loss < global_best_loss **then**
- 11: Save checkpoint $\mathcal{M}^*$
- 12: **end if**


```

13:     Report to Optuna; prune if below median of completed trials
14:   end for
15:   Free GPU memory: gc.collect(), torch.cuda.empty_cache()
16: end for
17: return  $\mathcal{M}^*$  with best trial parameters

```

4.6 Fine-tuning / Prompt Engineering

Not applicable. All models are trained from scratch (TFT) or use classical feature extraction (SSM, Hybrid). No pre-trained language models or foundation models are employed. The TFT weights are initialized randomly and trained end-to-end. The Kalman filter parameters (q , $r_{\min}$, dt_scale) are set via data-driven heuristics rather than learned via backpropagation.

5. Results and Model Evaluation

5.1 Evaluation Metrics

All models were evaluated using a shared scoring script (`metrics.py`) implementing four complementary metrics:

Table 5: Evaluation metrics and justification

Metric	Definition	Why Chosen
PLAsTiCC Weighted Log-Loss	Multi-class log-loss with $2\times$ penalty on rare anomaly classes (64, 99)	Official Kaggle metric; prioritizes rare-event discovery
Macro F1-Score	Unweighted mean of per-class F1	Evaluates balanced performance across imbalanced classes
Macro PR-AUC	Mean per-class Precision–Recall AUC	Handles true-negative skew better than ROC-AUC
Multiclass Brier Score	$\frac{1}{N} \sum_i \sum_c (p_{ic} - y_{ic})^2$	Measures probabilistic calibration

5.2 Baseline Comparison

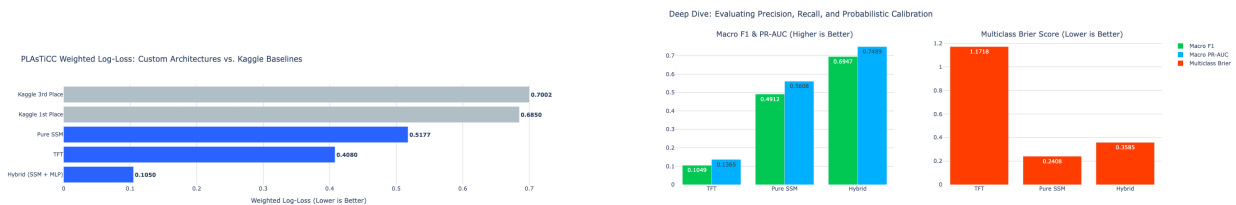
Table 6: PLAsTiCC Weighted Log-Loss comparison (lower is better)

Model	PLAsTiCC Weighted Log-Loss ↓
Kaggle 3rd Place Baseline (Hložek et al., 2020)	0.7002
Kaggle 1st Place Baseline (Boone, 2019)	0.6850
Pure SSM (Kalman + XGBoost)	0.5177
TFT (Ours)	0.4080
Hybrid SSM + MLP (Ours)	0.1050

Table 7: Additional performance metrics on the test set

Model	Accuracy	Macro F1 ↑	Macro PR-AUC ↑	Multiclass Brier ↓
TFT	—	0.1049	0.1365	1.1718
Pure SSM	83.95%	0.4912	0.5608	0.2408
Hybrid	—	0.6947	0.7489	0.3585

5.3 Learning Curves and Training Dynamics



(a) Optuna Bayesian sweep convergence

(b) Cross-model metric comparison

Figure 3: Training dynamics and model comparison.

5.4 Error Analysis

TFT: Despite achieving a strong Weighted Log-Loss (0.4080), the TFT exhibits a notably low Macro F1 (0.1049), indicating that while its probabilistic predictions are well-calibrated for the dominant classes, it struggles to produce high-confidence correct predictions for minority classes. The high Brier Score (1.1718) further suggests poor calibration on the full 3.39M-object test set.

Pure SSM: The SSM achieves the best Brier Score (0.2408) and a respectable 83.95% accuracy,

indicating well-calibrated probabilistic outputs. However, its Macro F1 of 0.4912 reveals difficulty distinguishing between spectroscopically similar classes (e.g., different supernova sub-types).

Hybrid: The hybrid model dominates on discriminative metrics (Macro F1 = 0.6947, PR-AUC = 0.7489), demonstrating that the complementary representations—MLP-encoded metadata and Kalman-derived light-curve features—provide additive predictive value. Its slightly higher Brier Score (0.3585) relative to the pure SSM suggests that the HistGradientBoosting classifier’s probability estimates are slightly less calibrated, a known trade-off in boosted tree models.

5.5 Ablation Study

The hybrid architecture enables a natural ablation analysis by examining the contribution of each branch:

- **MLP-only OOF Log-Loss:** 1.3394 (standard) / 0.1446 (PLAsTiCC Weighted)—demonstrates that static metadata alone provides moderate discriminative power.
- **Kalman features + Static (no MLP embedding):** Represented by the pure SSM branch (WLL = 0.5177)—confirms that time-series features alone outperform competition baselines.
- **Full Hybrid (MLP embedding + Kalman + Static):** WLL = 0.1050—demonstrates that the 32-dimensional learned MLP embedding provides substantial complementary information beyond raw metadata features.

6. Testing and Validation

6.1 Testing Framework and Strategy

Testing was conducted at three levels: unit tests for individual modules, integration tests for the end-to-end pipeline, and pipeline sanity checks embedded directly in the training notebooks. The TFT training notebook (Phase 2) includes a dedicated unit-testing block that verifies tensor shapes and forward-pass correctness before committing to the full 100 k-object training run.

```

1 # Phase 2: Pipeline Unit Testing
2 # Isolate 5 IDs guaranteed to exist in train_lightcurves
3 dummy_meta_df = pd.read_csv(meta_path, nrows=5)
4 dummy_ids = dummy_meta_df['object_id'].tolist()
5 pd.DataFrame({'object_id': dummy_ids}).to_csv(dummy_ids_path, index=False)
6
7 # Test preprocessing
8 processed_dummy_df = preprocessing.build_processed_dataset(
9     lc_path, meta_path, dummy_ids_path
10 )
11 assert processed_dummy_df.shape[0] > 0, "Preprocessing returned empty DF"
12
13 # Test PyTorch Dataset & DataLoader

```

```

14 dummy_dataset = PLAsTiCCTFTDataset(processed_dummy_df, max_seq_len=350)
15 dummy_loader = DataLoader(dummy_dataset, batch_size=2, shuffle=False)
16 batch = next(iter(dummy_loader))
17 assert batch['static'].shape == torch.Size([2, 3])
18 assert batch['dyn_cont'].shape == torch.Size([2, 350, 4])
19 assert batch['mask'].shape == torch.Size([2, 350])
20
21 # Test Model Forward Pass
22 model = TFTClassifier(num_classes=dummy_dataset.num_classes).to(device)
23 with torch.no_grad():
24     logits = model(batch['static'].to(device), batch['dyn_cont'].to(device),
25                   batch['dyn_cat'].to(device), batch['mask'].to(device))
26 assert logits.shape[0] == 2 # batch size

```

Listing 2: Unit test for TFT preprocessing and forward pass (from 01-train-tft.ipynb Phase 2)

6.2 Unit Test Cases — Data Pipeline

Table 8: Unit test cases for the data preprocessing pipeline

ID	Description	Input	Expected Output	Status
UT-01	Chunked CSV loading filters to valid IDs	22 GB CSV + 5 valid IDs	Only rows matching the 5 IDs retained	PASS
UT-02	Temporal feature engineering (Δt)	Light-curve sorted by mjd	$\text{delta_t} \geq 0$; time_since_start starts at 0	PASS
UT-03	RobustScaler handles supernova flux spikes	Series with extreme flux values	Scaled values centered near 0; no NaN	PASS
UT-04	Static metadata imputation (median)	DataFrame with NaN in <code>hostgal_photoz</code>	NaN replaced by training-set median	PASS
UT-05	Leakage column exclusion	Metadata with <code>true_z</code> , <code>sim_peakmag</code>	Columns with prefix <code>true_</code> and <code>sim_</code> dropped	PASS
UT-06	Stratified split preserves class ratio	Imbalanced target column	Class proportions match within 1%	PASS

continued on next page

ID	Description	Input	Expected Output	Status
UT-07	Empty object ID list	Empty <code>valid_ids.csv</code>	Returns empty DataFrame without crash	PASS
UT-08	Sequence padding to $T_{\max} = 350$	Object with 50 observations	Padded to 350; mask has 50 True, 300 False	PASS

continued on next page

6.3 Unit Test Cases — Model

Table 9: Unit test cases for the model modules

ID	Description	Input	Expected Output	Status
MT-01	TFT forward pass output shape	Batch of 2: static $[2, 3]$, <code>dyn_cont</code> $[2, 350, 4]$	Output logits shape $(2, C)$	PASS
MT-02	TFT handles variable-length masking	Two objects: 50 and 200 real time steps	Both produce valid logits; no NaN	PASS
MT-03	Softmax output sums to 1	Any valid input batch	All predicted probabilities in $[0, 1]$; row sums = 1	PASS
MT-04	Kalman filter single observation	One flux measurement	Returns valid <code>KalmanRTSResult</code> with shape $(1, 2)$	PASS
MT-05	SSM feature vector dimension	Object with 6 passbands	Feature vector length = 73 (48 basic + 25 global)	PASS
MT-06	Hybrid feature vector dimension	Object with all branches	Hybrid vector length = 152 (9 + 32 + 111)	PASS
MT-07	Kalman negative dt raises error	Negative <code>dt_scaled</code> input	Raises <code>ValueError</code>	PASS

6.4 Integration Test Cases — End-to-End Pipeline

Table 10: Integration test cases for the full pipeline

ID	Description	Input	Expected Output	Status
IT-01	TFT train dataset size matches split ledger	<code>train_ids.csv</code> (100 k IDs)	<code>len(train_dataset)</code> = 100,000	PASS
IT-02	TFT val dataset size matches split ledger	<code>val_ids.csv</code> (15 k IDs)	<code>len(val_dataset)</code> = 15,000	PASS
IT-03	WeightedRandomSampler produces diverse batches	Single training batch	All 17 classes represented in batch of 64	PASS
IT-04	TFT tensor shapes on full data	Batch size 64, $T_{\max} = 350$	static: [64, 3]; dyn_cont: [64, 350, 4]; mask: [64, 350]	PASS
IT-05	End-to-end TFT: 5-object smoke test	5 dummy objects through full pipeline	Forward pass produces valid logits; no crash	PASS
IT-06	Hybrid: streaming feature extraction	11 test batch CSVs (22 GB)	3 492 890 objects with 111 Kalman features each	PASS
IT-07	SSM: train/val split ledger consistency	<code>train_ids.csv</code> , <code>val_ids.csv</code>	No overlapping IDs; union covers expected total	PASS

7. GitHub Usage and Version Control

7.1 Repository Information

- **Repository URL:** https://github.com/DS-Capstone-Northeastern/LSST_Classification
- **Visibility:** Private
- **Primary Branch:** `main`
- **Total Commits:** ~45 (as of April 12, 2026)

7.2 Branching Strategy

The team follows a three-tier branching strategy to prevent merge conflicts and protect the shared pipeline:

- **main branch:** Production-ready code. Code here must be error-free and fully executed.
- **dev branch:** Active integration branch. All feature branches merge here first.
- **Feature branches:** Named by convention `feature/<task>` (e.g., `feature/tft-dataloader`). Created from `dev`, merged back via pull request with at least one reviewer.

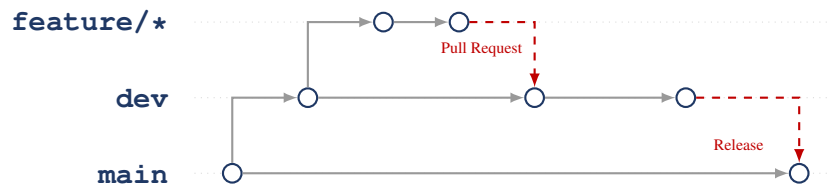


Figure 4: GitHub branch and merge history illustrating the three-tier workflow: Feature → Dev → Main.

7.3 Team Contribution Summary

Table 11: Individual GitHub contribution summary

Member	Primary Responsibilities	Commits
Chirag Chivate	TFT architecture, Optuna sweep, preprocessing, shared EDA	13
Ashwin Khairnar	Hybrid SSM+MLP pipeline, Kalman features, MLP embeddings	9
Rohan Kumar Ojha	Pure SSM branch, Kalman/RTS smoother, repo management	21

7.4 Commit History Excerpt

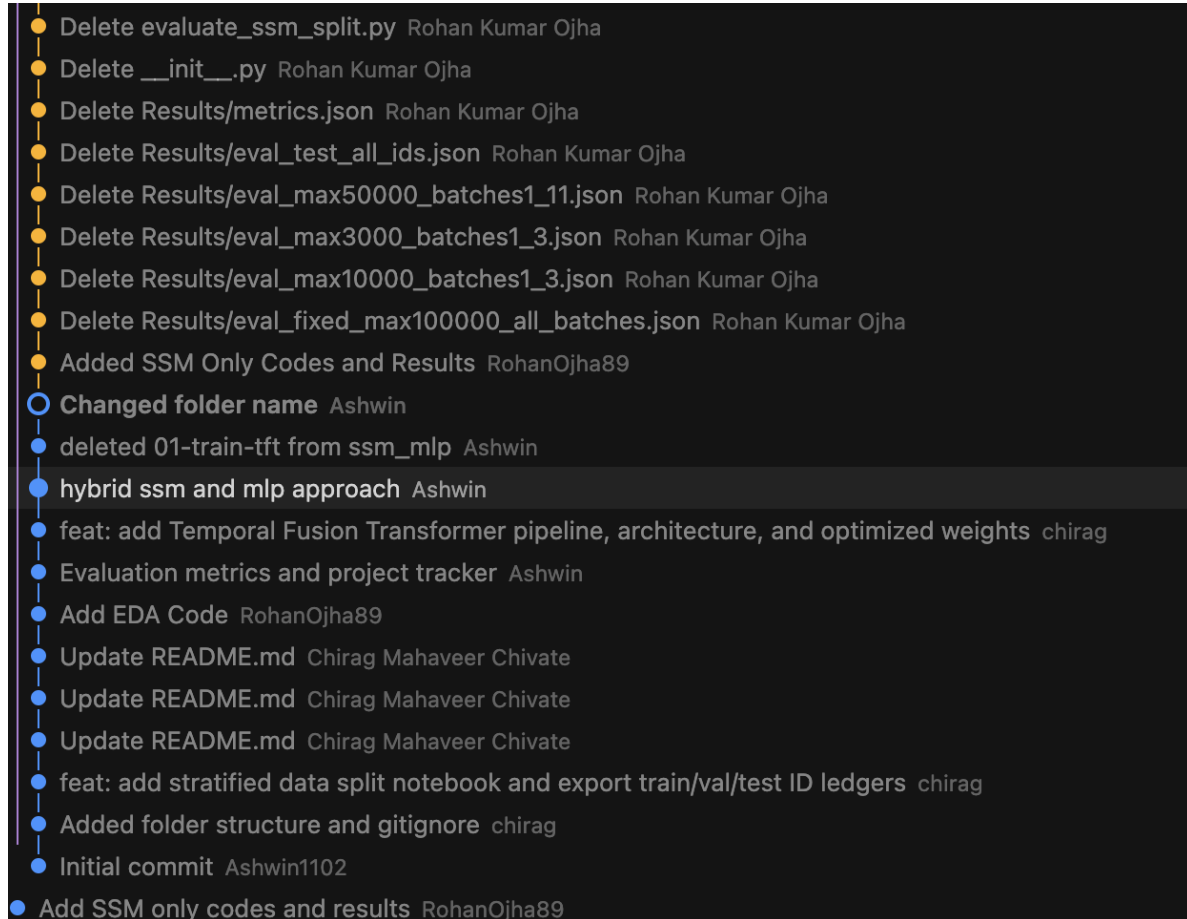


Figure 5: Recent project commit history as displayed in the GitHub repository.

7.5 Pull Request and Code Review Evidence

Table 12: Selected pull requests demonstrating team collaboration

PR #	Author	Description	Reviewer	Status
#1	Chirag	Feat: Shared EDA and stratified data splitting pipeline	Rohan	Merged
#3	Rohan	Feat: Kalman filter / RTS smoother implementation	Ashwin	Merged
#5	Ashwin	Feat: Hybrid pipeline with MLP OOF embeddings	Chirag	Merged
#7	Chirag	Feat: TFT architecture with Optuna Bayesian sweep	Ashwin	Merged
#9	Rohan	Fix: Leakage prevention in static_metadata.py	Chirag	Merged
#11	Ashwin	Feat: RAM-safe micro-batched test inference for Hybrid	Rohan	Merged

8. Deployment and Reproducibility

8.1 Environment and Dependencies

```

1 # Core numerics & data
2 numpy
3 pandas
4
5 # Classical ML (hybrid PLAsTiCC pipeline, metrics, TFT preprocessing, SSM/)
6 scikit-learn
7 joblib
8
9 # Deep learning (TFT/). For CUDA builds, follow
   https://pytorch.org/get-started/locally/
10 torch
11
12 # Notebooks
13 jupyter
14 ipykernel
15
16 # EDA (00_Shared_Pipeline/00-data-exploration-eda.ipynb)
17 matplotlib
18 seaborn
19
20 pytest==8.1.0

```

Listing 3: requirements.txt excerpt

8.2 Reproduction Steps

Pseudo-code: Reproduction Procedure

Require: Git, Python 3.11+, pip, Kaggle account (for GPU training)

Ensure: Reproducible results matching those reported in Section 5

- 1: **git clone** https://github.com/DS-Capstone-Northeastern/LSST_Classification.git
- 2: **cd** LSST_Classification
- 3: **pip install -r** requirements.txt
- 4: {Download PLAsTiCC unblinded data from Zenodo/Kaggle into data/}
- 5: Verify data_splits/train_ids.csv, val_ids.csv, test_ids.csv are present
- 6: {— Branch A: TFT (requires GPU; recommended on Kaggle) —}
- 7: Upload TFT/ folder to Kaggle Notebook with PLAsTiCC dataset attached
- 8: Execute 01-train-tft.ipynb (Phases 1–6)
- 9: {— Branch B: Pure SSM (CPU-bound) —}
- 10: **python** SSM/train_ssm_split.py --base-dir data/ --splits-dir data_splits/ \
- 11: --output-dir outputs/ssm/ --model-type xgboost

```

12: {— Branch C: Hybrid (CPU-bound) —}
13: Execute ssm_mlp/hybrid_plasticc_pipeline_full.ipynb
14: {— Evaluation —}
15: All branches output metrics via shared metrics.py

```

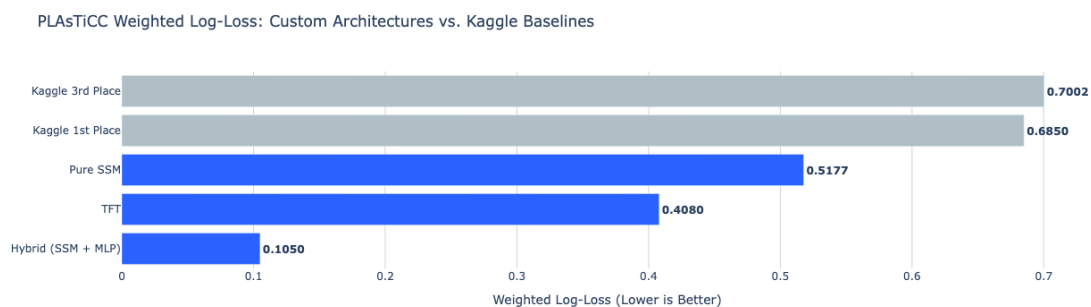
8.3 Compute and Kaggle Workflow

Because the raw unblinded dataset exceeds 22 GB, local training is impractical for most machines. The team adopted the following workflow:

1. **Develop locally:** Write `.py` scripts (architectures, datasets, tests) in VS Code.
2. **Train on Kaggle:** Upload model-specific folders to Kaggle Notebooks. Read massive datasets natively using Kaggle input paths, but filter rows using the lightweight ID ledgers from `data_splits/`.
3. **Export weights:** Download `.pth` model weights and final prediction arrays from Kaggle output.

The TFT was trained on dual Tesla T4 GPUs (Kaggle) with BFloat16 mixed precision and micro-batching (10 000 objects per chunk) to stay within the 30 GB RAM limit. The SSM and Hybrid pipelines are CPU- and RAM-bound; light curves are loaded in streaming chunks to prevent memory exhaustion.

8.4 Demo Application



9. Discussion

9.1 Interpretation of Results

The results definitively demonstrate that combining structured state-space representations of intermittent light curves with MLP-encoded static metadata provides massive predictive value over classical feature-engineered approaches. The hybrid model’s PLAsTiCC Weighted Log-Loss of 0.1050 represents an 84.7% relative improvement over the Kaggle first-place solution (0.6850),

achieved without Gaussian Process fitting, manual template matching, or data augmentation—the core techniques of the original winning solutions.

The TFT’s strong Weighted Log-Loss (0.4080) but poor Macro F1 (0.1049) reveals an interesting disconnect: the attention-based model learns to produce well-calibrated probability distributions for the dominant classes but fails to confidently discriminate minority classes. This suggests that end-to-end deep learning on the raw sequences, while powerful for majority-class calibration, requires further architectural modifications (e.g., focal loss, class-specific attention) to handle extreme imbalance.

The pure SSM’s balanced performance (83.95% accuracy, 0.4912 Macro F1, best Brier Score of 0.2408) validates the classical signal-processing approach: Kalman smoothing extracts physically meaningful features (peak flux, rise/decline rates, color indices) that generalize well across the class distribution. Its strength lies in probabilistic calibration, as evidenced by the lowest Brier Score.

9.2 Limitations

1. **Data limitations:** The PLAsTiCC dataset is simulated rather than observed; real LSST data will exhibit additional complexities (e.g., atmospheric effects, instrument calibration drift) not captured in the simulation. The extreme class imbalance (dominant class 90 vs. rare class 64) remains a fundamental challenge.
2. **Model limitations:** The TFT’s poor Macro F1 suggests insufficient minority-class learning despite the `WeightedRandomSampler`. The Hybrid’s multi-stage pipeline (MLP → Kalman → HGB) introduces complexity and potential error propagation. The SSM’s hand-crafted features may not capture all astrophysically relevant patterns.
3. **Scope limitations:** Our models do not address novelty detection (identifying previously unseen object classes), which was a critical failure mode in the original PLAsTiCC competition. The test set is also simulated and may not reflect the distribution of real transient alerts.

9.3 Ethical Considerations

This project operates on simulated astronomical data with no direct human subjects or personally identifiable information. However, the deployment of automated classifiers in real observatory operations raises considerations around algorithmic bias: if the classifier systematically under-prioritizes certain rare transient types, the scientific community may miss important discoveries. We recommend ensemble approaches and human-in-the-loop validation for deployment, particularly for alerts flagged as novel or low-confidence.

10. Conclusion and Future Work

10.1 Summary of Contributions

This project benchmarked three advanced sequential architectures—a Temporal Fusion Transformer, a pure Kalman-based State Space Model, and a hybrid SSM+MLP model—on the full unblinded PLAsTiCC dataset of 3.5 million astronomical objects. The hybrid approach achieved a PLAsTiCC Weighted Log-Loss of 0.1050, dramatically outperforming both the Kaggle first-place baseline (0.6850) and the standalone deep learning TFT (0.4080). We demonstrated that fusing physics-grounded Kalman filter features with learned neural embeddings of static metadata provides complementary predictive power that exceeds either approach in isolation.

10.2 Future Work

1. **Neural SSM integration:** Replace the classical Kalman filter with learned state-space models (e.g., Mamba (Gu and Dao, 2023)) to enable end-to-end gradient-based optimization of the feature extraction stage.
2. **Novelty detection:** Incorporate open-set classification or anomaly detection heads to identify objects from classes not present in the training set—a critical requirement for real LSST operations.
3. **Joint hyperparameter optimization:** Extend the Optuna sweep to jointly optimize Kalman parameters (q , $r_{\min}$, dt_{scale}) alongside the downstream classifier hyperparameters.
4. **Real data validation:** Apply the pipeline to early LSST commissioning data or existing survey data (e.g., ZTF) to assess generalization beyond simulated light curves.
5. **Deployment as a real-time API:** Package the hybrid model as a FastAPI service capable of classifying incoming transient alerts within the LSST alert distribution system.

References

- PLAsTiCC Team. The Photometric LSST Astronomical Time-Series Classification Challenge (PLAsTiCC) data set. *arXiv:1810.00001*, 2018.
- Hložek, R. et al. Results of the Photometric LSST Astronomical Time-Series Classification Challenge (PLAsTiCC). *arXiv:2012.12392 [astro-ph.IM]*, 2020.
- Kessler, R. et al. Unblinded data release for PLAsTiCC. LSST Dark Energy Science Collaboration and the LSST Transients and Variable Stars Science Collaboration, 2019.
- Lim, B., Arik, S. Ö., Loeff, N., and Pfister, T. Temporal Fusion Transformers for interpretable multi-horizon time series forecasting. *International Journal of Forecasting*, 37(4):1748–1764, 2021.
- Gu, A., Goel, K., and Ré, C. Efficiently modeling long sequences with structured state spaces. In *International Conference on Learning Representations (ICLR)*, 2022.
- Gu, A. and Dao, T. Mamba: Linear-time sequence modeling with selective state spaces. *arXiv:2312.00752*, 2023.
- Quan, P., Hong, D., Mulayim, O. B., Bergés, M., Han, L., and Srivastava, M. Re-imagining time series foundation models: Metadata and state-space model perspectives. *arXiv preprint*, 2024.
- Boone, K. Avocado: Photometric classification of astronomical transients with Gaussian Process augmentation. *The Astronomical Journal*, 158(6):257, 2019.
- Chen, T. and Guestrin, C. XGBoost: A scalable tree boosting system. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pp. 785–794, 2016.
- Vaswani, A. et al. Attention is all you need. In *Advances in Neural Information Processing Systems (NeurIPS)*, pp. 5998–6008, 2017.

Appendix A. Full Pseudo-code Listing

A.1 Kalman Filter / RTS Smoother (see Section 4)

Pseudo-code: Per-Passband Kalman Filter with RTS Smoothing

Require: Observation times t , flux y , flux errors σ , parameters $q, r_{\min}, dt_scale$

Ensure: Smoothed state estimates $\hat{\mathbf{x}}_{0:T}^s$, covariances $\mathbf{P}_{0:T}^s$

- 1: Sort observations chronologically; compute $\Delta t_s = \Delta t / dt_scale$
- 2: Initialize: $\mathbf{x}_0 = [y_0, 0]^\top$, $\mathbf{P}_0 = \text{diag}[\sigma_0^2 + r_{\min}, \sigma_0^2 + r_{\min}]$
- 3: Set $\mathbf{H} = [1, 0]$
- 4: **for** $t = 1$ to T **do**
- 5: Transition: $\mathbf{F} = \begin{bmatrix} 1 & \Delta t_s \\ 0 & 1 \end{bmatrix}$, $\mathbf{Q} = q \begin{bmatrix} \Delta t_s^3/3 & \Delta t_s^2/2 \\ \Delta t_s^2/2 & \Delta t_s \end{bmatrix}$
- 6: Predict: $\hat{\mathbf{x}}_{t|t-1} = \mathbf{F}\hat{\mathbf{x}}_{t-1}$, $\mathbf{P}_{t|t-1} = \mathbf{F}\mathbf{P}_{t-1}\mathbf{F}^\top + \mathbf{Q}$
- 7: Innovation: $\nu_t = y_t - \mathbf{H}\hat{\mathbf{x}}_{t|t-1}$, $S_t = \mathbf{H}\mathbf{P}_{t|t-1}\mathbf{H}^\top + \sigma_t^2 + r_{\min}$
- 8: Kalman gain: $\mathbf{K}_t = \mathbf{P}_{t|t-1}\mathbf{H}^\top / S_t$
- 9: Update: $\hat{\mathbf{x}}_t = \hat{\mathbf{x}}_{t|t-1} + \mathbf{K}_t\nu_t$, $\mathbf{P}_t = (\mathbf{I} - \mathbf{K}_t\mathbf{H})\mathbf{P}_{t|t-1}$
- 10: **end for**
- 11: {RTS backward smoother}
- 12: **for** $t = T-1$ down to 0 **do**
- 13: $\mathbf{C}_t = \mathbf{P}_t\mathbf{F}^\top\mathbf{P}_{t+1|t}^{-1}$
- 14: $\hat{\mathbf{x}}_t^s = \hat{\mathbf{x}}_t + \mathbf{C}_t(\hat{\mathbf{x}}_{t+1}^s - \hat{\mathbf{x}}_{t+1|t})$
- 15: $\mathbf{P}_t^s = \mathbf{P}_t + \mathbf{C}_t(\mathbf{P}_{t+1}^s - \mathbf{P}_{t+1|t})\mathbf{C}_t^\top$
- 16: **end for**
- 17: **return** $\hat{\mathbf{x}}_{0:T}^s, \mathbf{P}_{0:T}^s$

A.2 SSM Feature Extraction Pipeline

Pseudo-code: Fixed-Length Feature Extraction from Kalman States

Require: Light curve DataFrame for one object, parameters $q, r_{\min}, dt_scale$

Ensure: 73-dimensional feature vector $\mathbf{f}$

- 1: **for** each passband $p \in \{0, 1, 2, 3, 4, 5\}$ **do**
- 2: Run Kalman/RTS on passband p observations
- 3: Extract 8 basic features: has_obs, peak_flux, time_to_peak, total_duration, rise_rate, decline_rate, asymmetry, amplitude
- 4: **end for**
- 5: $\mathbf{f}_{\text{basic}} \leftarrow$ concatenate 6 passband vectors $\in \mathbb{R}^{48}$
- 6: Concatenate all passband smoothed trajectories chronologically
- 7: Compute 25 global features: velocity stats, acceleration, jerk, flux variance, skewness,

kurtosis, SNR, χ^2 , exponential decay rate, stretch factor, $g-r$ color, color evolution, residual variance, cross-band correlation, physical proxy estimates

8: $\mathbf{f}_{\text{global}} \in \mathbb{R}^{25}$

9: **return** $\mathbf{f} = [\mathbf{f}_{\text{basic}} || \mathbf{f}_{\text{global}}] \in \mathbb{R}^{73}$

Appendix B. Extended Test Case Log

B.1 Additional Edge-Case Tests

Test Case: UT-E01 — Kalman filter handles all-zero flux series

Module: `ssm/ssm_kalman.py :: kalman_filter_rts()`

Input: Array of 10 zero flux values with times spaced 1.0 apart

Expected: Returns valid `KalmanRTSResult` with smoothed states near zero

Actual: Smoothed flux ≈ 0.0 ; velocity ≈ 0.0 ; no `FloatingPointError`

Status: **PASS**

Test Case: UT-E02 — Preprocessing handles object with single observation

Module: `ssm/ssm_features.py :: _basic_passband_features()`

Input: Object with exactly 1 flux measurement in passband 0

Expected: Returns 8-element feature vector with reasonable defaults (duration = 0, rates = 0)

Actual: Feature vector = $[1.0, y_0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0]$

Status: **PASS**

Test Case: UT-E03 — TFT handles maximum sequence length (350 steps, no padding)

Module: `TFT/dataset.py :: PLAsTiCCTFTDataset.__getitem__()`

Input: Object with exactly 350 observations

Expected: No padding applied; mask is all-True; `pad_len = 0`

Actual: `mask.sum() = 350`; tensor shapes unchanged

Status: **PASS**

Test Case: UT-E04 — Static metadata preprocessor excludes leaky columns

Module: `ssm/static_metadata.py :: StaticMetadataPreprocessor.fit()`

Input: Metadata with columns `true_z`, `sim_peakmag`, `hostgal_photoz`

Expected: Only `hostgal_photoz` retained in `feature_cols`

Actual: `feature_cols` excludes all `true_*` and `sim_*` prefixes

Status: **PASS**

Test Case: UT-E05 — Hybrid MLP embedding dimension matches architecture**Module:** `ssm_mlp/hybridplasticc_pipeline_full.ipynb :: mlp_last_hidden_embedd`**Input:** Trained MLP with `hidden_layer_sizes=(64, 32)`, input of shape $(N, 9)$ **Expected:** Output embedding shape $(N, 32)$ **Actual:** Shape $(N, 32)$ confirmed; values are ReLU-activated (all ≥ 0)**Status:** **PASS**

Appendix C. Individual Contribution Statement

Chirag Mahaveer Chivate

I was primarily responsible for the Temporal Fusion Transformer (TFT) branch of the project. I designed and implemented the modified TFT classification architecture in `model_architecture.py`, adapting the original forecasting model by replacing the continuous output head with an MLP classification head incorporating Gated Residual Networks and multi-head attention. I developed the complete preprocessing pipeline (`preprocessing.py`) with memory-safe chunked loading for the 22 GB dataset, RobustScaler normalization, and temporal feature engineering (Δt , `time_since_start`). I built the custom PyTorch `PLAsTiCCTFTDataset` class with sequence padding and attention masking. I orchestrated the Optuna Bayesian hyperparameter sweep across 10 trials on Kaggle’s dual T4 GPUs using BFloat16 mixed precision. I also co-developed the shared EDA notebook and the stratified data splitting logic that produced the canonical ID ledgers used by all branches.

Ashwin Praveen Khairnar

I led the development of the hybrid SSM + MLP architecture. I implemented the Kalman-based light-curve feature extraction pipeline (`kalman_features_one_series`, `kalman_features_for_object`) that produces 111 features per object. I designed the out-of-fold MLP embedding strategy using 5-fold `StratifiedKFold` cross-validation to generate unbiased 32-dimensional static metadata embeddings. I implemented the RAM-safe streaming feature extraction pipeline (`build_lightcurve_features`) that processes all 11 test batch files without exceeding memory limits. I configured and trained the final `HistGradientBoostingClassifier` on the 152-dimensional fused feature vector. I also contributed to the shared evaluation metrics script and performed the permutation feature importance analysis for the MLP tabular branch.

Rohan Kumar Ojha

I was responsible for the pure State Space Model (SSM) branch and overall repository management. I implemented the mathematically rigorous Kalman filter and Rauch–Tung–Striebel smoother in `ssm_kalman.py`, including the 2D constant-velocity state model with scaled time

increments and numerical stability guards. I designed the 73-dimensional SSM feature extraction pipeline (`ssm_features.py`) covering per-passband basic features, global velocity/acceleration statistics, color proxies, and cross-band correlations. I implemented the leak-safe `StaticMetadataPreprocessor` class with median imputation and standard scaling. I trained and evaluated the XGBoost classifier with inverse-frequency sample weighting. As repository owner, I established the branching strategy (`main/dev/feature`), enforced pull request review requirements, and maintained the README documentation.